Programming Assignment: Tomasulo Algorithm with Multi-Issue

1. Summary

I extended the provided Python Tomasulo simulator (ROB, RAT, reservation stations, load/store queue with memory disambiguation) so that it can **issue up to 1-4 instructions per cycle, in order**. The issue width is read from the input file (`Issue width = N`, default 1) and can be overridden with `-w N`. While testing I found and fixed several bugs in the original code that made results wrong in some programs even at issue width 1 (section 3). All final register and memory values match an in-order reference interpreter on 6 directed tests and 250 random programs at every issue width (1000 simulator runs, 0 mismatches).

2. What I added

Issue stage (`issue.py`). The old `issue()` that handled one instruction was renamed `issue_one()` and now returns whether the instruction was issued. A new `issue(..., issue_width)` calls it up to `issue_width` times per cycle, in program order. Issue stops for the cycle when

- an instruction cannot issue because the ROB, the reservation stations of its unit, or the load/store queue is full (a structural hazard blocks everything behind it, so issue stays in order), or
- an instruction was a branch: the existing design stalls fetch until the branch resolves in EX, so nothing younger than a branch is fetched.

Instructions issued in the same cycle see each other's renamed destinations because the RAT is updated sequentially, so dependences inside an issue group are handled correctly (e.g. a `Mult.d` and the `Add.d` that uses its result can issue in the same cycle).

Commit stage (`commit.py`). Commit was rewritten as a loop that retires up to `commit_width` (= issue width) instructions per cycle, in order. Without it a 4-wide front end would just fill the ROB behind a 1-per-cycle commit. A branch now commits only after it has finished EX, and branch rows print their commit cycle.

Input parsing (`config.py`, new) and `main.py`. The simulator now reads the input format from the handout: hardware table (reservation stations, EX cycles, MEM cycles), `ROB entries`, `Issue width`, register and memory initial values, and the program. Input is case-insensitive and commas are optional. `main.py` prints the instruction status table, all registers, the non-zero memory words, and a `SUMMARY` line (cycle in which the last instruction commits). `Beq` was added next to `Bne`, `R0` is hardwired to zero, and `Ld`/`Sd` operands such as `-12(R10)` are parsed with a regular expression (the old code only handled one-digit register numbers).

3. Bugs found in the original code (and fixed)

Found by comparing against the reference interpreter on random programs.

1. **RAT overwritten by an older instruction (`wb.py`).** A broadcast always wrote its value into the RAT. If a younger instruction had already renamed the same register (WAW) and a slower older one broadcast later, the younger mapping was destroyed and later readers got the wrong value. The RAT is now updated only if it still points at the broadcasting ROB entry.
2. **Simulation ended before the last store reached memory (`main.py`).** The loop stopped when the ROB was empty, but a store leaves the ROB when its memory access starts, so the final write was lost. The loop now also waits for the memory unit to go idle.
3. **Loop counter clobbered in store-to-load forwarding (`mem.py`).** The loop variable `index` was reused inside the loop body, which caused an `IndexError` or skipped queue entries. The inner variable was renamed.
4. **Store allowed into memory too early (`mem.py`).** `check_if_sd_committable` was never actually called (the function object itself was tested, which is always true), so a store could write memory before older instructions had finished. It is now called, and a store may start only when it is at the head of the ROB (or right behind a head that already broadcast). Its original body also referenced a non-existent field (`ldsd_tag`).
5. **A branch could commit before finishing EX (`commit.py`)** when the integer unit takes more than one cycle.

Because of fixes 4 and 5 and the multi-commit loop, the exact cycle numbers at width 1 differ slightly from the original code in some rows (e.g. the original committed up to two instructions in some cycles even at width 1). The last-commit cycle for the supplied test is the same, 43.

4. Test cases

All tests are in `tests/`; each is checked against `tests/ref.py`, a simple in-order interpreter. Branch offsets are in bytes (`-12` = 3 instructions back), as in the supplied test.

Test	Purpose
`test_case.txt` (supplied)	loop with load/multiply/add/store and a branch
`t1_independent.txt`	12 independent int/FP instructions: best case for wide issue
`t2_dependent_chain.txt`	long RAW chains: little parallelism
`t3_memory.txt`	stores and loads to the same addresses: memory disambiguation, forwarding
`t4_loop.txt`	array-sum loop: branch every 5 instructions
`t5_structural.txt`	1 reservation station per unit, 4-entry ROB: structural stalls
250 random programs	random mix of ops, bounded loops and forward branches; random hardware parameters

5. Results: cycles until the last instruction commits

Test	width 1	width 2	width 3	width 4	speedup (4 vs 1)	result correct
test_case.txt	43	37	37	36	1.19	yes
t1_independent	22	16	15	15	1.47	yes
t2_dependent_chain	27	25	24	24	1.13	yes
t3_memory	30	30	30	30	1.00	yes
t4_loop	61	58	56	56	1.09	yes
t5_structural	23	22	22	22	1.05	yes
250 random programs (total)	15623	14106	13801	13672	1.14	all 1000 runs match

Why the gains are modest. Wider issue only helps the front end. t3 gets no gain because it is limited by the single memory port and the dependent store/load chain. t4 and the supplied test are limited by branches (fetch stalls until each branch resolves) and by the 15-cycle multiplier on the critical path. t5 is limited by its tiny reservation stations and ROB. Only t1, which has independent instructions, gets close to 1.5x, and even there the single CDB (one result per cycle) and one functional unit per type cap it.

Supplied test, issue width 1

	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[9]	[10]
Mult.d F4 F2 F20	[2]	[10, 24]	[]	[25]	[26]
Ld F6 0(R2)	[3]	[4, 4]	[8, 12]	[14]	[27]
Add.d F6 F4 F6	[4]	[26, 29]	[]	[30]	[31]
Sd F6 0(R2)	[5]	[6, 6]	[32, 36]	[]	[32]
Addi R1 R1 -4	[6]	[7, 7]	[]	[8]	[34]
Addi R2 R2 -4	[7]	[8, 8]	[]	[10]	[35]
Bne R1 R0 -12	[8]	[9, 9]	[]	[]	[36]
Addi R1 R1 -4	[10]	[11, 11]	[]	[12]	[37]
Addi R2 R2 -4	[11]	[12, 12]	[]	[13]	[38]
Bne R1 R0 -12	[12]	[13, 13]	[]	[]	[39]
Addi R1 R1 -4	[14]	[15, 15]	[]	[16]	[40]
Addi R2 R2 -4	[15]	[16, 16]	[]	[17]	[41]
Bne R1 R0 -12	[16]	[17, 17]	[]	[]	[42]
Add.d F20 F2 F2	[18]	[19, 22]	[]	[23]	[43]

Supplied test, issue width 2

	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6 0(R2)	[2]	[3, 3]	[8, 12]	[15]	[25]
Add.d F6 F4 F6	[2]	[25, 28]	[]	[29]	[30]
Sd F6 0(R2)	[3]	[4, 4]	[31, 35]	[]	[31]
Addi R1 R1 -4	[3]	[4, 4]	[]	[5]	[32]
Addi R2 R2 -4	[4]	[5, 5]	[]	[6]	[33]
Bne R1 R0 -12	[4]	[6, 6]	[]	[]	[33]
Addi R1 R1 -4	[7]	[8, 8]	[]	[9]	[34]
Addi R2 R2 -4	[7]	[8, 8]	[]	[10]	[34]
Bne R1 R0 -12	[8]	[10, 10]	[]	[]	[35]
Addi R1 R1 -4	[11]	[12, 12]	[]	[13]	[35]
Addi R2 R2 -4	[11]	[12, 12]	[]	[14]	[36]
Bne R1 R0 -12	[12]	[14, 14]	[]	[]	[36]
Add.d F20 F2 F2	[15]	[16, 19]	[]	[20]	[37]

Supplied test, issue width 4

	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[10]	[11]
Mult.d F4 F2 F20	[1]	[11, 25]	[]	[26]	[27]
Ld F6 0(R2)	[1]	[2, 2]	[8, 12]	[14]	[27]
Add.d F6 F4 F6	[1]	[27, 30]	[]	[31]	[32]
Sd F6 0(R2)	[2]	[3, 3]	[33, 37]	[]	[33]
Addi R1 R1 -4	[2]	[3, 3]	[]	[4]	[34]
Addi R2 R2 -4	[2]	[3, 3]	[]	[5]	[34]
Bne R1 R0 -12	[2]	[5, 5]	[]	[]	[34]
Addi R1 R1 -4	[6]	[7, 7]	[]	[8]	[35]
Addi R2 R2 -4	[6]	[7, 7]	[]	[9]	[35]
Bne R1 R0 -12	[6]	[9, 9]	[]	[]	[35]
Addi R1 R1 -4	[10]	[11, 11]	[]	[12]	[35]
Addi R2 R2 -4	[10]	[11, 11]	[]	[13]	[36]
Bne R1 R0 -12	[10]	[13, 13]	[]	[]	[36]
Add.d F20 F2 F2	[14]	[15, 18]	[]	[19]	[36]

Final state (identical at all widths): R2 = 20, F2 = 1, F4 = 3, F6 = 7, F20 = 2, other registers 0; memory

```
Addresses   Values
4           3
8           2
12          1
24          6
28          5
32          7
```

In the width-2 and width-4 tables several instructions share an ISSUE cycle (up to the width) and several commit in the same cycle; the first loop iteration's `Bne` delays the second group because fetch stalls until it resolves.

6. What works and what does not

Works: issue widths 1-4; correct results (registers and memory) on all tests; register renaming, ROB, load/store forwarding and memory disambiguation as in the original code; `Ld`, `Sd`, `Add`, `Addi`, `Sub`, `Add.d`, `Sub.d`, `Mult.d`, `Beq`, `Bne`.

Limitations:

- **No branch prediction.** The handout describes a one-bit predictor with an 8-entry BTB and squash/recovery, but the supplied code has none: it stalls fetch after every branch until the branch resolves in EX. I kept that behaviour (it is always correct) and did not add a predictor, so branches limit the speedup.
- **Branch offsets** are interpreted in bytes (offset/4 instructions) to match the supplied test (`-12`), not in words as the handout's formula suggests.
- Commit width is tied to the issue width; there is one CDB, one functional unit per type, and the memory is 256 bytes. The "# of FUs" column of the input is read but ignored (always 1).
- Memory addresses must be word-aligned and below 256.

7. How to run

```
cd code
python3 main.py test_case.txt -w 2
python3 ../tests/run_directed.py
python3 ../tests/run_random.py 250
```